Arch Technologies

Sharpening your hidden skills for a brighter future

Data Science Internship

Month 1 Submission Report

Submitted By: Sawaira Mumtaz
Internship Domain: Data Science
Email Address: sawairamumtaz369@gmail.com
Phone Number: 03155069089
Submission Date: August 30, 2026
Submitted To: Arch Technologies

1. Overview

This report documents the completion of the two Month 1 tasks assigned under the Data Science internship track at Arch Technologies: Titanic Survival Classification and Stock Price Prediction. Both tasks involved data pre-processing, model training, and performance evaluation using Python in Google Colab.

Task 1: Titanic Survival Classification

Objective: Build a machine learning classification model to predict whether a Titanic passenger survived, based on features such as passenger class, sex, age, and fare.

Dataset note: This implementation uses the Titanic dataset built into the Seaborn library (sns.load_dataset('titanic')), which contains the same passenger data as the Kaggle Titanic dataset. This source was used to avoid a Kaggle account/API setup step, while keeping the underlying data identical.

Approach: Rows with missing values were dropped, the categorical 'sex' feature was numerically encoded, and the data was split 80/20 into training and test sets. A Logistic Regression classifier was trained on pclass, sex, age, and fare to predict the 'survived' label.

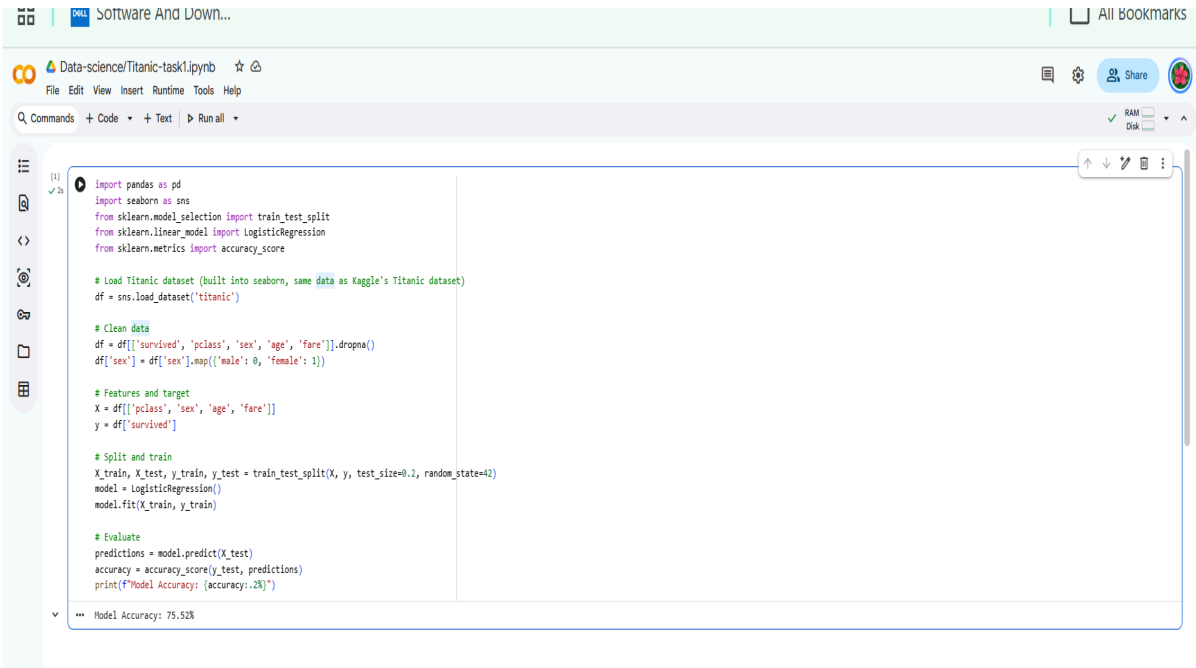
Code

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load Titanic dataset (built into seaborn, same data as Kaggle's Titanic dataset)
df = sns.load_dataset('titanic')

# Clean data
df = df[['survived', 'pclass', 'sex', 'age', 'fare']].dropna()
df['sex'] = df['sex'].map({'male': 0, 'female': 1})

# Features and target
X = df[['pclass', 'sex', 'age', 'fare']]
y = df['survived']
```

Split and train
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression()
model.fit(X_train, y_train)

# Evaluate
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print(f"Model Accuracy: {accuracy:.2%}")

Output



Result: The model achieved an accuracy of 75.52% on the held-out test set.

Task 2: Stock Price Prediction

Objective: Build a model to predict future stock prices using historical stock data, including opening price, closing price, high, low, and trading volume.

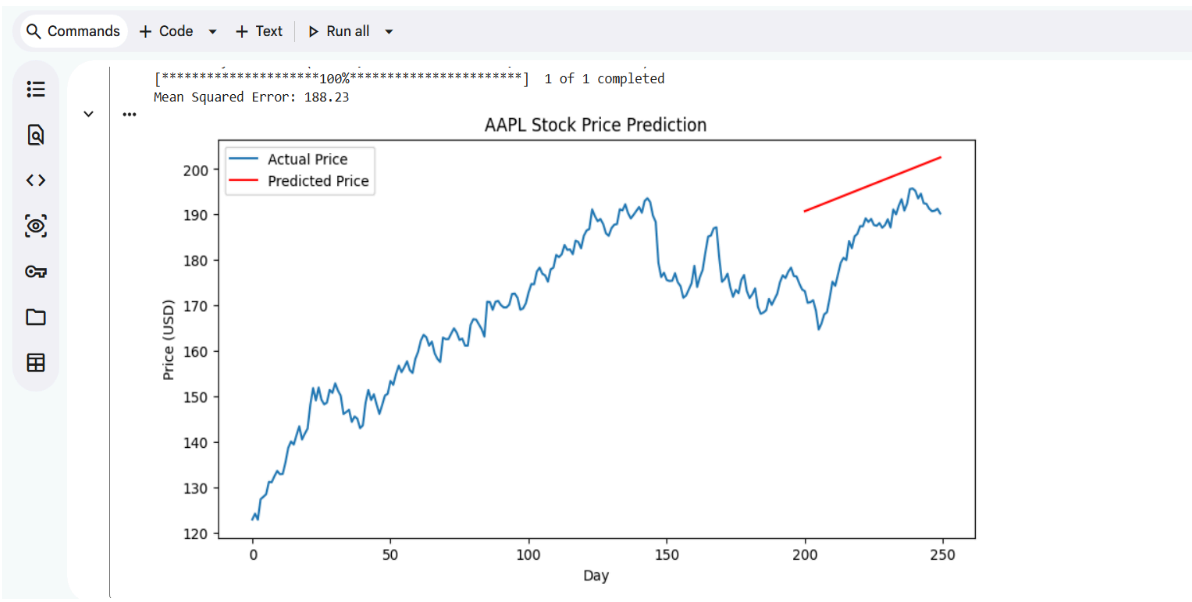
Approach: Historical daily closing prices for Apple Inc. (AAPL) from January 2023 to January 2024 were downloaded using the y finance library. The day index was used as the predictor feature and closing price as the target. A Linear Regression model was trained on the first 80% of the data (chronologically) and evaluated on the remaining 20%, as permitted by the task brief, which allows either Linear Regression or LSTM.

Code

```
!pip install yfinance -q
import yfinance as yf
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Download historical stock data (Apple)
data = yf.download('AAPL', start='2023-01-01', end='2024-01-01')
data = data[['Close']].reset_index()
data['Day'] = np.arange(len(data))

# Features and target
X = data[['Day']]
y = data['Close']

# Split (last 20% as test)
split = int(len(data) * 0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Train
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

# Evaluate
mse = mean_squared_error(y_test, predictions)
print(f"Mean Squared Error: {mse:.2f}")

# Plot
plt.figure(figsize=(10,5))
plt.plot(data['Day'], data['Close'], label='Actual Price')
```

```
plt.plot(X_test['Day'], predictions, label='Predicted Price', color='red')
plt.xlabel('Day')
plt.ylabel('Price (USD)')
plt.title('AAPL Stock Price Prediction')
plt.legend()
plt.show()
```

Output



Result: The model produced a Mean Squared Error (MSE) of 188.23 on the test set. The plot shows the actual AAPL closing price (blue) against the model's predicted trend (red) over the test period.

1. Notes and Limitations

Task 1 uses Seaborn's built-in Titanic dataset as a substitute for a manual Kaggle download, as noted above; the data itself is the standard Titanic dataset.

Task 2 uses Linear Regression rather than LSTM. Linear Regression was chosen for interpretability and fast training time; it models the general price trend rather than short-term fluctuations, which is reflected in the MSE and the plotted prediction line.

1. Conclusion

Both Month 1 tasks were completed successfully: a classification model for Titanic survival prediction (75.52% accuracy) and a regression model for AAPL stock price prediction (MSE 188.23). These tasks demonstrated core data science skills including data cleaning, feature preparation, model training, and performance evaluation.

LinkedIn Post :